PHENIKAA UNIVERSITY

PHENIKAA SCHOOL OF COMPUTING

COURSE: SOFTWARE ARCHITECTURE

TOPIC: Online movie streaming platform (Nozie platform)

Instructor: **Vũ Quang Dũng**

Group 03: Nghiêm Đức Việt (23010636)

Class Section: CSE703110-1-2-25(N02)

Ha Noi, Feb 2026

Table of Contents

ASSIGNED ROLE: BACKEND DEVELOPER

Main Tasks: Microservices Implementation & Database Design

1. AUTHENTICATION & AUTHORIZATION (Identity Service)

1.1 API Endpoints - Authentication

HTTP Method	Endpoint	Description	Access Level
POST	/api/auth/register	Register a new account	Public
POST	/api/auth/login	Login via username/password	Public
POST	/api/auth/logout	Logout and invalidate tokens	Secured
POST	/api/auth/refresh	Obtain a new access token via refresh token	Public
GET	/api/auth/me	Retrieve current logged-in user info	Secured
GET	/api/auth/sessions	List active sessions	Secured
DELETE	/api/auth/sessions/{id}	Terminate a specific session	Secured

1.2 Use Cases - Identity Domain

Use Case	Actor	Flow Description
UC1 – Register	Customer	1. User submits registration details. 2. Identity Service creates account in DB. 3. Publishes UserRegisteredEvent. 4. Returns success to Client.
UC2 – Login via Credentials	Customer	1. User inputs email/password. 2. Identity Service validates hash. 3. If valid, issues JWT Access & Refresh Tokens.

		4. Client stores tokens securely.
UC2_1 – Social Auth Login	Customer	1. User selects "Login with Google/Facebook". 2. Client handles OAuth flow with Provider. 3. Identity Service verifies Provider Token. 4. Service creates/links account and issues App JWT.
UC2_2 – Two-Factor Auth	Customer	1. After initial credential check, System requests 2FA code. 2. User enters code (SMS/Authenticator). 3. Identity Service validates code. 4. Issues final Access Token.
UC3 – Logout & Revoke	Customer	1. User selects Logout. 2. Client deletes local token. 3. Client calls API to blacklist Refresh Token on server.
UC4 – Change Password	Customer	1. User provides old and new passwords. 2. Identity Service validates old password. 3. Updates DB with new hash. 4. Triggers security notification.
UC5 – Manage Sessions	Customer	1. User requests list of active sessions. 2. Identity Service returns list of devices/IPs.

		3. User can revoke unrecognized sessions.

1.3 Security Implementation

- **JWT Management**: Manage Access Token (short-lived) & Refresh Token (long-lived)
- **Centralized Security**: Centralized at API Gateway with JWT validation
- **Token Blacklisting**: Support token revocation upon logout
- **Password Hashing**: Use BCrypt for password hashing

2. CRUD OPERATIONS

2.1 Movie Service (MongoDB)

Applying **Database per Service** pattern with MongoDB:

Use Case	Description
Browse Catalog	Retrieve movie list with pagination
Advanced Search	Search by multiple criteria (Actor, Year, Genre)
Watch Movie	Stream movie (after subscription verification)
Add to Watchlist	Add movie to "watch later" list
Rate & Review	Rate and review movies

2.2 Customer Service - CRUD

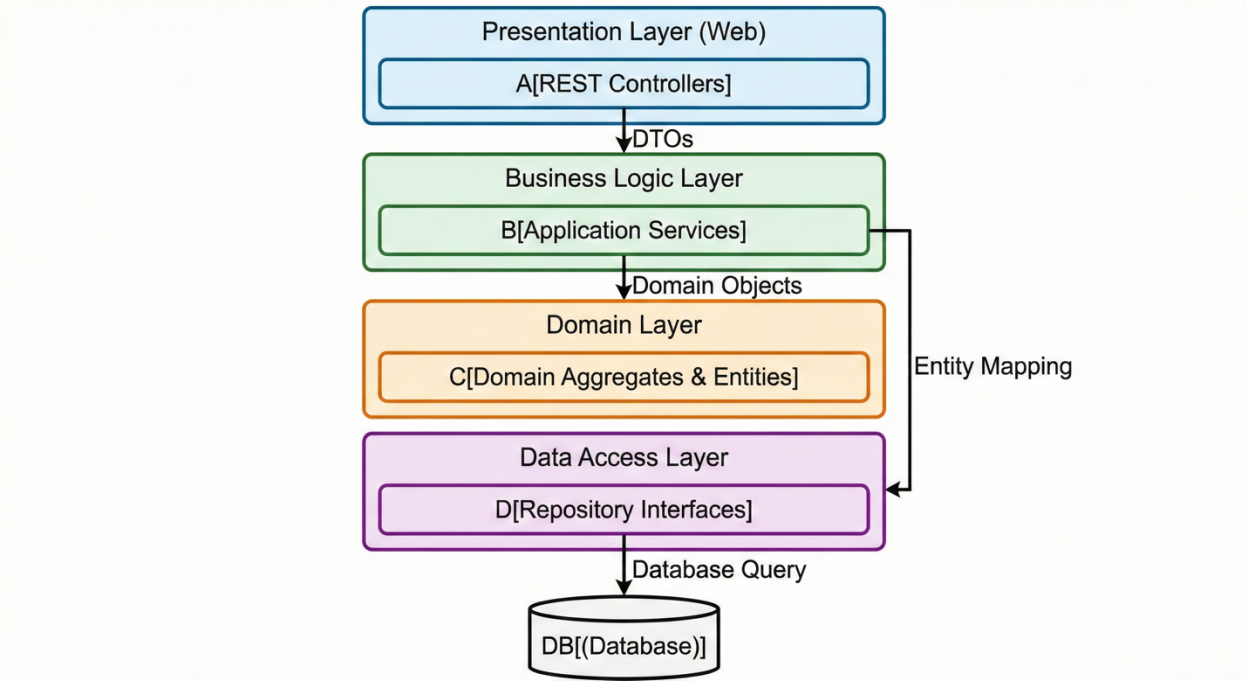
HTTP Method	Endpoint	Description
GET	/api/customers/user/{userId}	Retrieve customer profile & settings
GET	/api/customers/{id}/watchlist	Retrieve watchlist
POST	/api/customers/{id}/watchlist	Add movie to watchlist
GET	/api/customers/{id}/history	Retrieve viewing history
POST	/api/customers/{id}/history	Record viewing entry

2.3 Admin Service - User Management

HTTP Method	Endpoint	Description
GET	/api/admin/users	List all users (paginated)
PUT	/api/admin/users/{id}/status	Update account status (ACTIVE/BLOCKED)
GET	/api/admin/audit-logs	View admin activity logs

3. SYSTEM ARCHITECTURE

3.1 Layered Architecture



3.2 Database per Service Pattern

Service	Database	Purpose
Identity Service	PostgreSQL	User accounts, credentials, sessions
Customer Service	PostgreSQL	Customer profiles, preferences
Movie Service	MongoDB	Movie metadata, catalog
Payment Service	PostgreSQL	Subscriptions, transactions

3.3 API Gateway Routing

YAML

```
spring:
  cloud:
    gateway:
      routes:
        - id: identity-service
          uri: lb://identity-service
          predicates:
            - Path=/api/auth/**
        - id: movie-service
          uri: lb://movie-service
          predicates:
            - Path=/api/movies/**
          filters:
```

```
        - name: CircuitBreaker
          args:
            name: movieService
            fallbackUri: forward:/fallback/movie
      - id: customer-service
        uri: lb://customer-service
        predicates:
          - Path=/api/customers/**
```

4. TECHNOLOGIES USED

- **Framework**: Spring Boot 3.x
- **Security**: Spring Security + JWT
- **Service Discovery**: Netflix Eureka
- **API Gateway**: Spring Cloud Gateway
- **Databases**: PostgreSQL, MongoDB
- **Caching**: Redis
- **Build Tool**: Maven/Gradle